

REPORT

Rabbits and Recurrence Relations

by

Swetali Smruti Sikhya

Student

B.Sc. Biology | Aspiring Computational Biologist

1. Problem Overview

We begin with 1 pair of newborn rabbits. Every pair takes exactly one month to reach reproduction age. Once mature, each pair produces a litter of k new pairs every month, and rabbits never die.

Given positive integers n (≤ 40 months) and k (≤ 5 , the litter size), the goal is to determine the total number of rabbit pairs present after n months. This is a variant of the classical Fibonacci sequence, generalised to allow a litter size other than 1.

Given: Positive integers $n \leq 40$ and $k \leq 5$.

Return: The total number of rabbit pairs that will be present after n months, if we begin with 1 pair and in each generation, every pair of reproduction-age rabbits produces a litter of k rabbit pairs (instead of only 1 pair).

1.1. Sample Dataset

5 3

1.2. Sample Output

19

2. Approach

The population after each month can be expressed purely in terms of the population in the two preceding months, so the problem reduces to evaluating a linear recurrence relation. Rather than modelling individual rabbits, the approach tracks aggregate pair counts month by month:

- Every pair alive last month survives into this month (no deaths).
- Only pairs that were already mature two months ago are old enough to breed this month; each such pair contributes k new pairs.

Building the sequence bottom-up (from month 1 upward) avoids the redundant recomputation that a naive top-down recursive implementation would incur.

3. Algorithm

Let $F(n)$ denote the number of pairs present after n months:

$$F(n) = F(n-1) + k * F(n-2), \quad F(1) = F(2) = 1$$

When $k = 1$, this reduces to the standard Fibonacci sequence. The algorithm computes $F(n)$ iteratively:

1. Initialise two variables to the base cases $F(1) = 1$ and $F(2) = 1$
2. For each month from 3 to n , compute the new total as (last month's pairs) + $k \times$ (pairs from two months ago).
3. Slide the two tracked values forward by one month and repeat.

4. After the final iteration, the most recent value is $F(n)$, the answer.

4. Complexity Analysis

Table 1. Time and space complexity: naive recursion vs. iterative approach

Approach	Time	Space
Naive recursion (no memoisation)	$O(2^n)$	$O(n)$ (call stack)
Iterative / bottom-up (this solution)	$O(n)$	$O(1)$

The iterative solution runs in $O(n)$ time, computing each term exactly once, and $O(1)$ additional space, since only the last two values need to be retained at any point in the loop rather than the full sequence.

5. Python Implementation

Below is the finalized implementation. The recurrence is computed iteratively using two rolling variables rather than storing the full sequence, keeping memory use constant regardless of n .

```
"""
Rosalind Problem: FIB - Rabbits and Recurrence Relations

Given: Positive integers  $n \leq 40$  and  $k \leq 5$ .
Return: The total number of rabbit pairs present after  $n$  months, given that
we begin with 1 pair and that each pair of reproduction-age rabbits produces
a litter of  $k$  rabbit pairs (instead of only 1 pair, as in the classic
Fibonacci story).
"""

def fib_rabbits(n: int, k: int) -> int:
    """
    Compute the number of rabbit pairs alive after  $n$  months.

    Parameters
    -----
    n : int
        The number of months to simulate.
    k : int
        The litter size how many new pairs each mature (reproduction-age)
        pair produces every month.

    Returns
    -----
    int
        The total number of rabbit pairs after  $n$  months.

    Recurrence used
    -----
     $F(n) = F(n-1) + k * F(n-2)$ 
     $F(1) = F(2) = 1$ 

    Why this formula works:
    -  $F(n-1)$  counts every pair that was already alive last month.
      Rabbits never die in this problem, so all of them are still here.
    -  $k * F(n-2)$  counts the newborns this month. Only pairs that were
```

```

        already mature TWO months ago (i.e. alive at month n-2) are old
        enough to breed this month, and each such pair contributes k new
        pairs.
    """

    # Base cases: in months 1 and 2 there is just the single starting
    # pair. It isn't mature yet in month 2, so no offspring exist yet.
    if n <= 2:
        return 1

    # Instead of storing the whole sequence in a list (which would use
    # O(n) memory), we only ever need the last two values to compute the
    # next one. So we keep two "rolling" variables:
    # prev2 -> the value from two months ago, i.e. F(n-2)
    # prev1 -> the value from one month ago, i.e. F(n-1)
    prev2, prev1 = 1, 1 # F(1), F(2)

    # Walk forward from month 3 up to month n, updating the pair counts
    # one month at a time.
    for _ in range(3, n + 1):
        # Compute this month's total pairs using the recurrence:
        # current = (pairs alive last month) + k * (pairs mature enough to breed)
        current = prev1 + k * prev2

        # Slide the two-value window forward by one month:
        # what was "last month" (prev1) becomes "two months ago" (prev2)
        # the value we just computed becomes the new "last month" (prev1)
        prev2, prev1 = prev1, current

    # After the loop finishes, prev1 holds F(n) the answer.
    return prev1

def main():
    # Read n and k from the sample dataset file (two space-separated
    # integers on one line, e.g. "5 3").
    with open("sample_dataset.txt") as f:
        n, k = map(int, f.read().split())

    # Run the calculation and print the result, exactly as Rosalind
    # expects the answer to be submitted.
    result = fib_rabbits(n, k)
    print(result)

# This guard ensures main() only runs when the script is executed
# directly (e.g. `python fib_solution.py`), not when it's imported as a
# module elsewhere.
if __name__ == "__main__":
    main()

```

6. Result

The implementation was verified against the official Rosalind sample dataset ($n = 5$, $k = 3$), traced by hand month by month:

Table 2. Step-by-step computation of rabbit pairs for the sample dataset.

Month	Formula	Pairs
1	base case	1
2	base case	1
3	$F(2) + 3 \times F(1) = 1 + 3 \times 1$	4
4	$F(3) + 3 \times F(2) = 4 + 3 \times 1$	7
5	$F(4) + 3 \times F(3) = 7 + 3 \times 4$	19

For the sample input $n = 5$, $k = 3$, running `fib_solution.py` produces an output of 19 total rabbit pairs, matching the hand-traced values above and the official Rosalind sample output.

The actual dataset submitted to and accepted by Rosalind is unique to each user’s account; that personal dataset and its corresponding output are therefore omitted from this report and are not included in the accompanying repository

7. Skills Demonstrated

- Translating a word problem into a formal recurrence relation.
- Bottom-up dynamic programming to avoid exponential-time naive recursion.
- Space optimization via rolling variables instead of storing a full sequence array.
- Reasoning about the time and space complexity of an algorithm.
- Writing clear, well-commented, testable Python code with file I/O.

8. Conclusion

The rabbit population growth problem is efficiently solved by recognising it as a linear recurrence relation closely related to the Fibonacci sequence. An iterative, bottom-up implementation computes the exact pair count in linear time and constant space, correctly handling the upper bound of $n \leq 40$ months well within practical limits. This exercise reinforces a broader pattern applicable across many bioinformatics and algorithmic problems: identifying an underlying recurrence and solving it with dynamic programming rather than brute-force recursion.